

SYSTEM DESIGN · 분산 메시지 큐

분산 메시지 큐 설계

멋사 백엔드 파트 스터디



4장 핵심 설계

“많은 메시지를 빠르게 처리하면서, 장애가 나도 메시지를 안전하게 보관하려면 어떻게 설계해야 할까?”

1

Producer

메시지를 만들어 큐에 보냅니다

2

Queue

생산자·소비자를 직접 연결하지 않고
중간에서 전달합니다

3

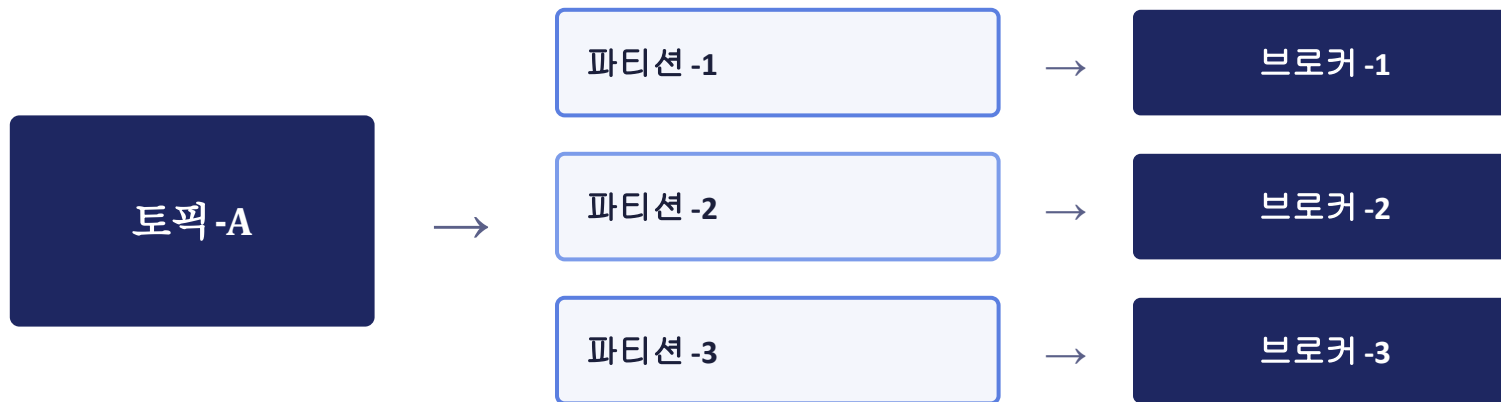
Consumer

자신의 상황에 맞게 큐에서 메시지를
꺼내 처리합니다

오늘은 여기서 시작해서 “대규모” 버전을 설계합니다→

토픽이 너무 커지면? → 파티션

문제: 하나의 토픽에 메시지가 몰리면, 서버 한 대가 병목이 된다



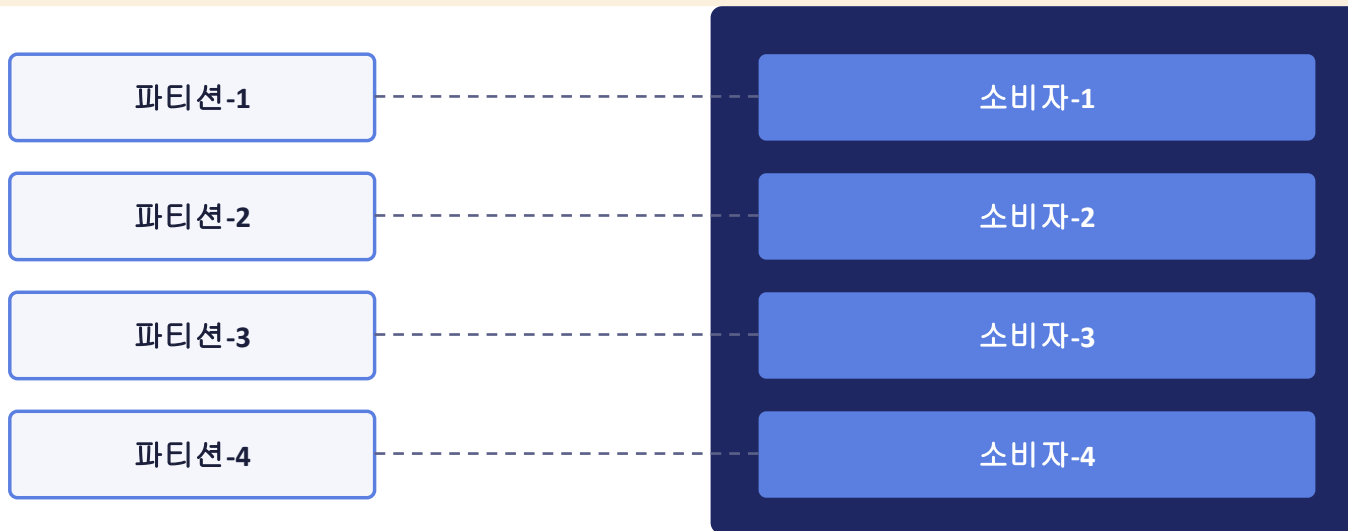
오프셋 (Offset)

파티션 안에서 메시지의 위치.
파티션 내부는 FIFO — 순서가 유지됨.
소비자는 오프셋으로 어디까지 읽었는지 관리.

핵심: 파티션은 처리량 — 즉 “데이터를 어떻게 나눠 저장·병렬 생산할까”의 문제를 푼다.

컨슈머 그룹: 소비를 병렬화하는 법

문제: 소비자가 하나뿐이면 여러 파티션을 순서대로 읽어야 해서 결국 소비 속도가 병목이 된다



규칙: 파티션 : 컨슈머 = 1 : 1 (그룹 내)

같은 그룹 안에서 같은 파티션을 여러 컨슈머가 읽으면 파티션 내부 순서가 소비 단계에서 깨집니다.

그래서 컨슈머는 파티션 수보다 늘려도 처리량이 늘지 않습니다.

정리: 파티션은 “저장·생산”의 확장성, 컨슈머 그룹은 “소비”의 확장성 — 서로 다른 축의 설계입니다.

컨슈머가 죽으면? → 리밸런싱

문제: 컨슈머 하나가 죽으면 그 컨슈머가 담당하던 파티션은 누가 처리할까?



같은 과정이 반대 방향으로도 일어납니다 — 새 컨슈머가 그룹에 합류할 때도 파티션을 다시 나눕니다.

그럼 실제로는 어떤 흐름으로 일어날까?

리밸런싱, 실제로는 이렇게 일어난다

시나리오 1 · 새 컨슈머 합류

- 1 처음엔 컨슈머 A 혼자 모든 파티션을 소비
- 2 컨슈머 B가 그룹 합류 요청
- 3 코디네이터가 전체에 재조정 필요 알림
- 4 리더 선출 → 새 파티션 배치 계획 수립
- 5 A, B 각자 배정된 파티션에서 소비 시작

시나리오 2 · 컨슈머 장애 이탈

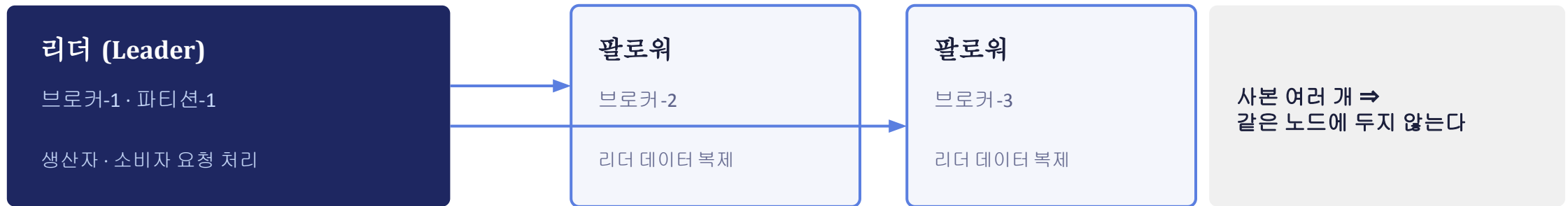
- 1 A, B 둘 다 코디네이터에 하트비트 전송 중
- 2 A에 장애 발생 → 하트비트 끊김
- 3 코디네이터가 일정 시간 뒤 장애로 판단
- 4 B가 재조정 요청 받고 그룹 재합류
- 5 남은 파티션 전부 B에게 재배포

컨슈머 수가 늘거나 줄 때마다 코디네이터가 감지하고 파티션을 다시 나눠주기 때문에 처리는 끝가지 않는다

그런데 컨슈머가 아니라 브로커 자체가 죽으면?

브로커가 죽으면? → 복제

문제: 파티션 데이터가 브로커 한 대에만 있다면, 그 브로커가 죽는 순간 데이터도 함께 사라진다



파티션은 처리량을 높이려 데이터를 나누는 방법, 복제는 장애에 대비해 같은 데이터를 여러 곳에 두는 방법 — 목적이 다릅니다.

그런데 복제를 여러 브로커에 걸쳐서 하다 보면, 어디까지 끝나야 생산자에게 "저장 성공" ACK를 보낼 수 있을까?

ISR과 ACK: 속도 vs 안전성

ISR (In-Sync Replicas)

리더와 동기화가 잘 되어 있는 사본들의 집합. 뒤처진 사본은 일시적으로 제외되고, 따라잡으면 다시 포함됩니다.

ACK = 0

응답을 기다리지 않음

가장 빠름
손실 위험 가장 큼

예: 로그 수집, 모니터링 지표

ACK = 1

리더 저장 시점에 응답

중간 속도
리더 장애 시 유실 가능

예: 알림, 사용자 활동 로그

ACK = all

ISR 전체 복제 후 응답

가장 느림
가장 안전함 (ISR 기준)

예: 결제, 주문, 재고 차감

실무 Tip: `min.insync.replicas` 설정으로 ISR 최소 개수를 강제하기도 합니다 — ISR이 0개면 ACK=all도 의미가 없으니깐요

몇 번 전달할 것인가

1

At-most-once

최대 한 번

✓ 중복 없음

△ 유실 가능

예: 센서 데이터, 시세 스트리밍

2

At-least-once

최소 한 번

✓ 유실 없음

△ 중복 가능

예: 알림, 주문 상태 업데이트

3

Exactly-once

정확히 한 번

✓ 유실·중복 없음

△ 구현 복잡도 ↑↑

예: 결제, 계좌 이체

유실 가능성 · 중복 가능성 · 구현 복잡도 — 시스템 요구사항에 맞게 선택합니다.

확장성 · 가용성 · 데이터 안전성의 균형

메시지가 많다

파티션으로 나눠 여러 브로커에서 처리

확장성

소비가 느리다

컨슈머 그룹으로 병렬 소비 (1:1 규칙)

확장성

컨슈머가 죽는다

리밸런싱으로 파티션 재배포

가용성

브로커가 죽는다

복제로 다른 노드에 사본 유지

지속성 · 내구성

언제 성공인가

ISR·ACK로 지연 vs 안전성 결정

지연 ↔ 안전성

몇 번 전달하나

at-most / at-least / exactly-once 선택

지연 ↔ 안전성